2/27/2026 Update

Active Learning Resume-from-Base Comparison (lr=1e-5, 15 epochs)

This table shows results from `test_al_resume.sh` with `freeze_backbone=False`, `lr=1e-5`, and `epoch=15`. All runs use `sub_ratio=0.3` and resume from `model_imagenet_resnet_base.pth` (base val acc: **0.7223**).

Strategy	Model Test ACC	Overall Val ACC Change	Status
Baseline	0.7162	0.718077	-
Random	0.7308	0.718077 → 0.720769 (+0.002692)	improved
Stratified	**0.7369**	0.718077 → 0.726154 (+0.008077)	improved
Coreset	0.7359	0.718077 → 0.719231 (+0.001154)	improved
Diversity	0.7362	0.718077 → 0.723846 (+0.005769)	improved
Uncertainty Stratified	0.7292	0.718077 → 0.719231 (+0.001154)	improved
Hierarchical	0.7346	0.718077 → 0.724615 (+0.006538)	improved
`Attribute`	**0.7369**	0.718077 → 0.722308 (+0.004231)	improved

Observations:

- **All strategies improved** over the baseline (0.718077) after fine-tuning with `lr=1e-5` for 15 epochs without backbone freezing.
- **Best Test ACC**: `Hybrid` (0.7400).
- **Best Val ACC improvement**: `Stratified` (+0.008077), benefiting from balanced class coverage during fine-tuning.
- **Uncertainty Stratified** performed below expectation (0.7292 Test ACC), possibly due to using a generic pretrained proxy model (ResNet18 weights, not the actual resumed checkpoint) to compute entropy scores, leading to noisy uncertainty estimates.
- **Attribute strategy** (with conditional rarity, R=0.01, B=5, class_balance=inf) achieved competitive performance (0.7369 Test ACC, +0.004231 val), demonstrating that metadata-driven selection can match geometry/diversity baselines when paired with proper fine-tuning LR.

Grid Search Results for Attribute Strategy Parameters (2/27/2026)

We conducted a comprehensive grid search over the parameters *R*, *P*, *C*, and the `use_entropy` toggle to understand their interactions and optimize the Attribute Strategy. The tests were run across 4 sub-ratios (`0.1`, `0.3`, `0.5`, `0.9`) on the `pinkcolor_fabrictexture` generated set.

Sub Ratio = 0.1

Scenario	Parameters	Test ACC	Val ACC
baseline_def	(R=0.01, P=5.0, C=inf, e=F)	**0.7300**	0.7146
strict_strat_entropy	(R=0.01, P=5.0, C=inf, e=T)	0.7215	0.7154
pure_greedy	(R=1.0, P=10.0, C=0.0, e=F)	0.7208	0.7154
soft_conservative_e0	(R=0.1, P=5.0, C=15.0, e=F)	0.7223	0.7146
soft_conservative_e1	(R=0.1, P=5.0, C=15.0, e=T)	0.7208	0.7138
balanced_exp_p5_c8	(R=0.3, P=5.0, C=8.0, e=T)	0.7223	0.7185
balanced_exp_p10_c8	(R=0.3, P=10.0, C=8.0, e=T)	0.7223	0.7200
aggro_exp_p5_c4	(R=0.8, P=5.0, C=4.0, e=T)	0.7223	0.7177
aggro_exp_p10_c4	(R=0.8, P=10.0, C=4.0, e=T)	0.7277	0.7200

Sub Ratio = 0.3

Scenario	Parameters	Test ACC	Val ACC
baseline_def	(R=0.01, P=5.0, C=inf, e=F)	0.7369	0.7262
strict_strat_entropy	(R=0.01, P=5.0, C=inf, e=T)	0.7346	0.7138
pure_greedy	(R=1.0, P=10.0, C=0.0, e=F)	0.7215	0.7162
soft_conservative_e0	(R=0.1, P=5.0, C=15.0, e=F)	0.7392	0.7200
soft_conservative_e1	(R=0.1, P=5.0, C=15.0, e=T)	0.7262	0.7115
balanced_exp_p5_c8	(R=0.3, P=5.0, C=8.0, e=T)	0.7362	0.7108
balanced_exp_p10_c8	(R=0.3, P=10.0, C=8.0, e=T)	0.7362	0.7092
aggro_exp_p5_c4	(R=0.8, P=5.0, C=4.0, e=T)	0.7362	0.7138
aggro_exp_p10_c4	(R=0.8, P=10.0, C=4.0, e=T)	**0.7415**	0.7115

Sub Ratio = 0.5

Scenario	Parameters	Test ACC	Val ACC
baseline_def	(R=0.01, P=5.0, C=inf, e=F)	**0.7438**	0.7123
strict_strat_entropy	(R=0.01, P=5.0, C=inf, e=T)	0.7415	0.7123
pure_greedy	(R=1.0, P=10.0, C=0.0, e=F)	0.7338	0.7138
soft_conservative_e0	(R=0.1, P=5.0, C=15.0, e=F)	**0.7438**	0.7208
soft_conservative_e1	(R=0.1, P=5.0, C=15.0, e=T)	0.7400	0.7162
balanced_exp_p5_c8	(R=0.3, P=5.0, C=8.0, e=T)	0.7308	0.7077
balanced_exp_p10_c8	(R=0.3, P=10.0, C=8.0, e=T)	0.7362	0.7100
aggro_exp_p5_c4	(R=0.8, P=5.0, C=4.0, e=T)	0.7400	0.7185
aggro_exp_p10_c4	(R=0.8, P=10.0, C=4.0, e=T)	0.7408	0.7131

Sub Ratio = 0.9

Scenario	Parameters	Test ACC	Val ACC
baseline_def	(R=0.01, P=5.0, C=inf, e=F)	0.7454	0.7223
strict_strat_entropy	(R=0.01, P=5.0, C=inf, e=T)	0.7438	0.7131
pure_greedy	(R=1.0, P=10.0, C=0.0, e=F)	0.7400	0.7192
soft_conservative_e0	(R=0.1, P=5.0, C=15.0, e=F)	0.7408	0.7146
soft_conservative_e1	(R=0.1, P=5.0, C=15.0, e=T)	0.7392	0.7192
balanced_exp_p5_c8	(R=0.3, P=5.0, C=8.0, e=T)	**0.7500**	0.7162
balanced_exp_p10_c8	(R=0.3, P=10.0, C=8.0, e=T)	0.7446	0.7138
aggro_exp_p5_c4	(R=0.8, P=5.0, C=4.0, e=T)	0.7431	0.7154
aggro_exp_p10_c4	(R=0.8, P=10.0, C=4.0, e=T)	0.7400	0.7177

Analysis

1. **The Danger of Pure Greedy**: The `pure_greedy` approach ($C = 0$) performs the worst across low evaluation ratios (0.1, 0.3). Without class-level Stratification quotas, throwing out all logic except Frequency Penalty and Rarity leads to performance drops in fine-tuning, emphasizing the value of our soft C score.
2. **Aggressive Exploration Shines ($R = 0.8$, $C = 4.0$)**: By loosening the absolute restriction on stratification ($C = 4.0$ instead of ∞) and massively scaling up Rarity tracking ($R = 0.8$), the `aggro_exp_p10_c4` scenario achieves extremely strong results across 0.3 and 0.5. The heavy frequency penalty ($P = 10$) adequately prevented the model from indexing too heavily on the synthesized injection flood.
3. **Entropy Tuning**: Dynamic entropy ($e = T$) does well in aggressive settings but causes occasional degradations in more conservative spaces (see `soft_conservative` comparisons).
4. **Baseline Comparison (Ratio=1.0)**: For reference, using **100% of the candidate data (Ratio=1.0)** yields a Test ACC of **0.7488**. Remarkably, the `balanced_exp_p5_c8` strategy at `Ratio=0.9` achieved an even higher Test ACC of **0.7500**, demonstrating that intelligent filtering can outperform training on the entirety of the noisy synthetic data!
5. **Overall Best Strategy**: Averaging across all sub-ratios, **`aggro_exp_p10_c4` ($R = 0.8$, $P = 10.0$, $C = 4.0$, Entropy=True)** represents the most robust and high-performing

parameter combination. It consistently dominates the low-to-mid ratio regimes (0.1 to 0.5) by aggressively seeking rare features while safely blocking flooded features and maintaining a soft class balance.

Conclusion: Tuning the Attribute Strategy to mimic `Coreset` + `Stratified` (via Soft C + High R) allows the strategy to exceed the conservative defaults! When allocating budgets, choosing a strong R and applying a compensatory P to manage spam achieves peak domain adaptability.

2/20/2026 Update

Active Learning Resume-from-Base Comparison

This table compares the test results generated by `test_al_resume.sh` against the provided baseline (`logs_imagenet/AL_attr_resume_base_resnet_imagenet_0.3_1000.log`). All tasks use `sub_ratio=0.3` and resume from the base model.

Strategy	Model Test ACC	Val Accuracy Difference	Status
Baseline	0.7162	0.718077	-
Random	0.7385	0.718077 -> 0.717692 (-0.000385)	declined
Stratified	0.7362	0.718077 -> 0.725385 (+0.007308)	improved
Coreset	0.7346	0.718077 -> 0.720000 (+0.001923)	improved
Uncertainty Stratified	0.7385	0.718077 -> 0.721538 (+0.003461)	improved
Diversity	0.7315	0.718077 -> 0.720769 (+0.002692)	improved
Hierarchical	0.7415	0.718077 -> 0.716154 (-0.001923)	declined
Hybrid	**0.7446**	0.718077 -> 0.717692 (-0.000385)	declined
`Attribute`	0.7369	0.718077 -> 0.720000 (+0.001923)	improved

Observations:

- **Best Test ACC**: `Hybrid` strategy achieved the highest raw Model Test Accuracy (0.7446).
- **Best Overall Accuracy Variation**: `Stratified` provided the best improvement in attribute/hard-case accuracy comparison (+0.007308).

Generation-Guided Active Learning (From Scratch) Comparison

This section compares the new `gen_probe` strategy against the baseline `random` and `stratified` strategies **from scratch** (i.e., without `--resume_model`). All runs use `sub_ratio=0.3`, 1000 generated images, and are trained for 30 epochs on ResNet-18.

Strategy	Model Test ACC	Status compared to Random
Baseline	0.2462	-
Random	0.5754	-
Stratified	**0.5908**	+0.0154
Gen-Probe	0.5600	-0.0154

Observations:

- The `gen_probe` strategy (Test ACC: 0.5600) underperformed both `random` (0.5754) and `stratified` (0.5908) when trained from scratch in this setup.
- While the motivation for `gen_probe` is strong (mining hard examples based on domain gap), the strategy might be selecting examples that are *too hard* or outliers, causing the model to struggle to learn generalizable features from scratch.
- Tuning the selection metric (e.g., mixing in some random/diversity samples, taking the middle uncertainty instead of absolute highest, or adjusting the `probe` model's training

epochs) might be required to make it surpass Stratified/Attribute strategies.

Low Budget Active Learning Comparison (ratios: 0.01, 0.02, 0.05)

To evaluate the strategies under extreme data constraints, further trials were conducted from scratch allocating only 1% to 5% of the source data budget along with the 1000 generated images.

Strategy	0.01 (104 samples)	0.02 (208 samples)	0.05 (520 samples)
Random	0.3323	0.3669	0.3977
Stratified	0.3154	0.3623	0.3931
Coreset	0.2846	0.3085	0.3246
Diversity	0.3269	0.3238	0.4162
Hybrid	**0.3323**	0.3662	**0.4169**
Attribute	0.3172	**0.3669**	0.3969

Observations:

- Under extreme constraints (≤ 2%), basic strategies like **Random** and **Attribute** hold the best ground, whereas geometry-dependent methods like Coreset collapse because the sample size is too sparse to cover the feature space.
- By 5%, the **Hybrid** and **Diversity** strategies begin extracting meaningful features, dominating the board (~0.416), while Coreset still lags behind.

---

3. Mathematical Formulation of Gen-Probe

The core idea of **Generation-Guided Active Learning (Gen-Probe)** is to use a model trained exclusively on generated images to act as a diagnostic “probe” on the original source data. This identifies the “domain gap”—the features present in real images that the generative model fails to synthesize.

Selection Mechanism:

1. **Probe Training**: We train a model M_{gen} exclusively on the pool of generated images (e.g., 1000 generated images).
2. **Hardness Scoring (Entropy)**: We evaluate M_{gen} on the entire **source training pool** (e.g., `train_idx_imagenet.npy` consisting of 10,400 unlabeled original images). For each image x_i, the probe outputs a probability distribution across C classes: $P(y|x_i)$. The score assigned to the image is its **Information Entropy** $H(x_i)$:
   $$ H(x_i) = - \sum_{c=1}^{C} P(y=c|x_i) \log P(y=c|x_i) $$
   - A **low entropy** ($H \to 0$) means M_{gen} is confident; the generated images already cover these features well.
   - A **high entropy** means M_{gen} is entirely confused; these images contain features distinct from the generated domain.
3. **Data Selection**: We sort all samples descending by $H(x_i)$ and select the top K images (e.g., 30% of the pool) that the probe model found most difficult.

---

4. Future Research Directions: Why did “Hard Mining” fail here?

The initial run of `gen_probe` yielded an accuracy of 0.5600, which is lower than `random` selection (0.5754). This is a well-known phenomenon in Active Learning literature: **Hard Example Mining is extremely susceptible to outliers.**

When a model is trained from scratch using *only* the Top-K highest entropy samples, it is essentially being trained on the most ambiguous, heavily occluded, or mislabeled images in the dataset. A model cannot build robust decision boundaries when given only edge cases.

Uncertainty-Weighted Stratified Strategy

The **Uncertainty Stratified** strategy aims to select the most informative samples (highest uncertainty) while strictly preserving a balanced class distribution in the selected subset.

1. Uncertainty Calculation (Entropy)

For a multi-class classification problem with C classes, let x_i be an unlabeled sample and $P(y = c|x_i)$ be the predicted probability of class c by the current model. The uncertainty score for sample i is calculated using Information Entropy:
$$ U_i = -\sum_{c=1}^{C} P(y=c|x_i) \log P(y=c|x_i) $$
A higher U_i indicates that the model is less confident about the prediction.

2. Stratified Quota Allocation

Let K be the total target number of samples to select, and $N_{classes}$ be the total number of unique ground-truth (or pseudo-label) classes in the candidate pool. The budget is distributed uniformly across all classes:
$$ K_{class} = \lfloor \frac{K}{N_{classes}} \rfloor $$
Any remainder samples $K \pmod{N_{classes}}$ are distributed iteratively to the first few classes until the budget is exhausted.

3. Stratified Selection

For each individual class c: 1. Isolate the subset of samples $\mathscr{I}_c$ that belong to class c. 2. Sort $\mathscr{I}_c$ in descending order based on their assigned uncertainty scores U_i. 3. Select the top K_{class} samples with the highest uncertainty from $\mathscr{I}_c$.

By combining point-wise entropy maximization with class-wise histogram equalization, this strategy avoids the common mode collapse in standard uncertainty sampling (where the model might exclusively select difficult samples from only one or two dominant/confusing classes).

Ablation Study for Attribute Parameters

The following table summarizes the ablation experiments with sub_ratio=0.7 for the AttributeFocusedStrategy parameters on ImageNet dataset. Each row corresponds to a different configuration where one or more parameters are set to specific values while others are zero (except for the baseline).

Configuration	Alpha (rarity_power)	Beta (uncertainty_boost)	Gamma (log_acc_scale)	Accuracy	Notes
Baseline (all zero)	0	0	0	0.699388	No attribute weighting
Traditional AL(Coreset)	-	-	-	0.69873	
Rarity only	1	0	0	0.700433	Only rarity component active
Uncertainty only	0	1	0	0.704842	Only uncertainty bonus active
					Only log-

Log weight only	0	0	1	0.709308	derived weights active
All	1	1	1	0.700769	
All, but scales	1	1	10	0.700384	

Note: Experiments run with ResNet model, sub_ratio=0.7, attributes=color,background.

Active Learning Strategy Usage Guide

Overview

I have implemented multiple Active Learning strategies for the HiBug project to address the poor performance of the original Core-set method.

Available Strategies

1. CoreSetStrategy (Original Method)

```
python alpred.py --al_strategy coreset --al_initial_labeled 400 --al_embedding_dim 256
```

- **Principle**: Selects samples farthest from labeled data (diversity-first)
- **Use Cases**: Uniform data distribution, need to maximize coverage
- **Issues**: May select outlier samples, disrupting class balance

2. StratifiedStrategy (Stratified Sampling)

```
python alpred.py --al_strategy stratified
```

- **Principle**: Stratified sampling by class proportion, maintaining class balance
- **Use Cases**: Imbalanced data, need representation from every class
- **Advantages**: Maintains original data distribution, simple and efficient
- **Speed**: Fastest (no feature extraction needed)

3. DiversityStrategy (Diversity Clustering)

```
python alpred.py --al_strategy diversity --al_clusters 100 --al_embedding_dim 256
```

- **Principle**: K-means clustering followed by representative sample selection from each cluster
- **Use Cases**: Data with natural clustering structure, need to cover different data patterns
- **Advantages**: Balances diversity and representativeness

4. HybridStrategy (Mixed Strategy)

```
python alpred.py --al_strategy hybrid --al_coreset_ratio 0.3 --al_stratified_ratio 0.5 --
  al_embedding_dim 256
```

- **Principle**: Combines advantages of multiple methods
  - 30% Core-set (diversity)
  - 50% Stratified (class balance)
  - 20% Random (randomness)
- **Use Cases**: Complex data distributions, need comprehensive consideration
- **Advantages**: Most robust, highly adaptive

5. AttributeFocusedStrategy (Attribute-Driven Sampling)

```
python alpred.py --al_strategy attribute --attribute_file save/attribute10.json \
  --al_attribute_focus color,background --al_attribute_rarity_power 1.5 --
  al_attribute_uncertainty_boost 1.0
```

- **Principle**: Builds a score for every unlabeled sample by combining attribute rarity, metadata-level uncertainty, and optional diagnostic weights derived from logs.
- **Use Cases**: Scenarios where human annotations describe attribute slices (color, background, hairstyle, etc.) and the model underperforms on specific combinations.
- **Advantages**: Lightweight (no feature extraction), controllable focus on high-value slices, deterministic once metadata is fixed.

The total score S_i for sample i is the sum across all attributes A of four components:
$S_i = \sum_{a \in A}(R_{i,a} + U_{i,a} + L_{i,a} + P_{i,a})$

Where: * $\boldsymbol{R_{i,a}}$ **(Class-Conditional Rarity)**: $\left(\frac{1}{f_{a}(v_{i,a} | y_i) + \varepsilon} \right)^{\alpha}$. Default $\alpha = 0.01$. Measures how rare an attribute value is *within its predicted class*. * $\boldsymbol{U_{i,a}}$ **(Uncertainty)**: β if marked uncertain in JSON, else 0. If `--al_attribute_use_entropy True` is set, this is dynamically replaced by Information Entropy $\beta \cdot (-\sum P \log P)$ from model inference. Default $\beta = 1.0$. * $\boldsymbol{L_{i,a}}$ **(Log Accuracy)**: $\gamma \cdot (1 - \text{ACC}_a(v_{i,a}))$. Default $\gamma = 0.0$. * $\boldsymbol{P_{i,a}}$ **(Frequency Penalty)**: $-\lambda \cdot \text{FreqRatio}(v_{i,a})$. Default $\lambda = 5.0$. This term continuously suppresses over-sampling from heavily flooded slices.

Samples are sorted by S_i (with a small injected noise term for tie breaking) and the top n_{select} indices are returned. When a sample exhibits multiple rare or low-accuracy attributes, the contributions accumulate, ensuring multi-attribute failures are prioritized.

Understanding the Frequency Penalty (*P*) and Generated Images While the formula for $P_{i,a}$ computes a simple frequency ratio, it perfectly neutralizes the bias introduced by injecting artificially generated data. Specifically, the ratio is calculated over the *combined* candidate pool:
$$\text{FreqRatio}(v) = \frac{N_{\text{base}}(v) + N_{\text{gen}}(v)}{\sum_{v'} \big(N_{\text{base}}(v') + N_{\text{gen}}(v') \big)}$$
Where N_{base} represents the counts in the original unlabeled dataset, and N_{gen} represents the newly injected synthetic bounding boxes (e.g., 1000 images focused purely on `color:pink` and `texture:fabric`). Because N_{gen} concentrates an enormous mass into a few specific attribute values (v), those specific values experience an artificially inflated FreqRatio. The $-\lambda \cdot \text{FreqRatio}$ penalty ensures that the active learning selector does not blindly exhaust its budget on the synthetic flood, forcing it to balance the newly injected data against naturally occurring hard cases.

The 2D Synergy: *R* (Class-Conditional) vs. *P* (Global) At first glance, Rarity (R) rewarding low frequency and Penalty (P) penalizing high frequency seem contradictory. However, because R evaluates *within a specific class* ($P(v|y)$) while P evaluates *across the entire global pool* (FreqRatio), they form a powerful 2D filter: * **High *R* + Low *P* (Class-rare, Global-rare)**: The most valuable “Hard Cases” (e.g., a “pink elephant”). They receive massive score boosts. * **High *R* + High *P* (Class-rare, Global-common)**: Features that are generic globally but strange for this specific class. R and P partially offset, preventing the model from over-indexing on globally redundant background textures just because they appeared in a new class. * **Low *R* + High *P* (Class-common, Global-common / Artificial Flooding)**: Boring redundant samples or the exact heavily-injected generated slices (e.g., 1000 synthetic `pink` dogs). The massive P penalty sinks their total S_i score, acting as an automatic spam filter to prevent them from exhausting the selection budget.

Optionally, enable an additional **Class Stratification Score (*C*)** via `--al_attribute_class_balance`. After computing S_i, the selector performs a greedy re-ranking that rewards classes whose current selections fall below their per-class quota. When the flag is set to a finite value $w > 0$, each class c keeps a running count n_c and target quota $t = n_{\text{select}}/N_{\text{classes}}$. The next candidate from class c receives an additive bonus C_c:
$$C_{c} = w \times \frac{t - n_c}{t}$$
which is added to its base score S_i inside `_soft_class_selection` (see `utils/active_learning.py`). Classes under their quota get positive bonuses, while classes at or above the quota see the term shrink toward zero or become negative, letting pure attribute scores win. Candidates with unknown labels (e.g., synthetic-only entries) sit in a separate queue and are inserted only when their base score exceeds every class-adjusted option, so unlabeled data does not distort the balancing pass.

During iteration k of `_soft_class_selection`, every labeled candidate i from class $c = y_i$ uses the effective score
$\tilde{S}_i^{(k)} = S_i + C_c^{(k)}$
where $C_c^{(k)} = w \times \frac{t - n_c^{(k)}}{t}$, and $n_c^{(k)}$ is the number of times class c has already been chosen. The algorithm picks the candidate with the largest $\tilde{S}_i^{(k)}$ (breaking ties with the injected noise from the base ranking), appends it to the selection list, and then updates $n_c^{(k+1)} = n_c^{(k)} + 1$ for that class. If the highest raw-score unlabeled item u satisfies $S_u > \max_i \tilde{S}_i^{(k)}$, the iteration chooses u instead and leaves all class counts unchanged, where S_u denotes the raw attribute score of the unlabeled candidate (no class bonus exists for unlabeled entries). That iteration simply appends u to `selected`. The loop stops once either $|\text{selected}| = n_{\text{select}}$ or all class and unlabeled queues are empty; in the latter case `_finalize_balanced_selection` backfills the remaining slots from the global ranking. "All other class and unlabeled queues are empty" literally means every per-class list (and the unlabeled list) has been drained—no candidate remains to evaluate—so the loop has no further element to pull even if some deficits are still positive.

This deficit-based soft balancing is a contribution inspired by proportional-fair and deficit round-robin schedulers: we convert each class shortfall $(t - n_c)$ into a temporary additive bonus while preserving the metadata-driven ordering within every class, providing a controllable bridge between pure attribute scoring and hard stratification. The motivation was empirical: attribute scores alone over-pick head classes, yet strict round-robin ignores the large score spread across classes, so we needed a lightweight middle ground that can lean toward balance without throwing away the metadata signal—a criterion none of the standard AL heuristics offered out of the box.

Setting `--al_attribute_class_balance inf` bypasses the soft bonus and triggers `_round_robin_class_selection`, the literal round-robin implementation. Each class queue contributes its current best-scoring sample, then the queue moves to the back. Exhausted queues are removed, and the process repeats until either the requested budget is filled or only unlabeled/leftover samples remain (at which point the code falls back to global sorting in `_finalize_balanced_selection`). This guarantees per-class counts differ by at most one (subject to availability) before any remaining slots revert to global scores.

Round-robin mechanics (as implemented in `_round_robin_class_selection`): 1. Build per-class queues sorted by the raw attribute score + noise. 2. Walk the queues in order; on each step pop the head of the current class and append it to `selected`. 3. Move that class to the back of the list; if the queue is empty, drop it permanently. 4. Repeat until `n_select` items are collected or every class queue is empty, then rely on `_finalize_balanced_selection` to fill any residual slots from the global score ranking. This produces the same effect as traditional stratified sampling but preserves the attribute ranking within each class.

Example (ImageNet, `sub_ratio=0.3`, ten classes): the quota is $t = 3,120/10 = 312$ samples per class. Running with `--al_attribute_class_balance 0` let the desk class drop to 133 samples (4.3 %) because it had few high attribute scores. Re-running the same configuration with `--al_attribute_class_balance 4` gave each desk candidate an early bonus of $4 \times (312 - 133)/312 \approx 2.29$, so they entered the greedy loop sooner. The final desk count rose to 273 samples (8.8 %), while large classes such as backpack and purse were nudged down toward the quota. Desk growth stopped at 273 because the global budget $n_{\text{select}} = 3,120$ was filled once the bonus had already reduced the biggest deficits; the algorithm cannot insert additional desk samples after the budget is exhausted even though $n_{\text{desk}} < t$. This illustrates how finite weights gently pull distributions toward balance without forcing perfect equality.

Weight tuning versus sub-ratio: because $C_c = w \times (t - n_c)/t$ scales with the quota $t = n_{\text{select}}/N_{\text{classes}}$, a fixed w becomes weaker as the sub-sampling ratio grows. To preserve roughly the same maximum bonus seen at `sub_ratio = 0.3`, scale w in proportion to t: $w_{\text{new}} \approx w_{0.3} \times (t_{\text{new}}/312)$. For ImageNet's ten classes this gives practical anchors: - `sub_ratio = 0.7` (quota ≈ 728): use $w \approx 9$–10. - `sub_ratio = 0.9` (quota ≈ 936): use $w \approx 12$.

Smaller weights still work but apply weaker nudges; larger weights push the distribution closer to stratified counts. Set `inf` if the run truly needs round-robin behavior.

The selector still picks the same total budget—$(10,400 \times 0.3 = 3,120)$ samples in this setup. Turning on `--al_attribute_class_balance 4` does not add or remove items; it just changes which indices fill those 3,120 slots. `_finalize_balanced_selection` appends the highest remaining base scores until the list has exactly n_{select} entries, ensuring the class-aware re-ranking is purely a reordering step.

Workflow - Provide an attribute JSON (for example `save/attribute10.json`) that maps each attribute to per-sample values and optional uncertain indices. - Optionally restrict attention to a subset via `--al_attribute_focus` (comma-separated keys). - (Optional) Supply a diagnostic log with `--al_attribute_log_path`; each `Rare Case` entry contributes a failure weight (1 − ACC) through the parser in `utils.active_learning.parse_attribute_priority_log`. - Tune `--al_attribute_rarity_power`, `--al_attribute_uncertainty_boost`, and `--al_attribute_log_acc_scale` to balance between metadata-driven rarity, known uncertain slices, and empirical accuracy signals. - (Optional) Provide pre-counted generated samples via `--al_attribute_extra_counts color:pink,texture:fabric`; when counts are omitted they default to `--num_added_img`, and if the flag itself is omitted the selector now parses `--attributes` (tokens such as `pinkcolor` or `fabrictexture`) to auto-inject the same `--num_added_img` count for those attribute-value pairs.

Because the method operates purely on metadata, it integrates cleanly with the rest of the pipeline—no embedding extraction or model forward passes are required during selection.

Parameter Description

General Parameters

- `--al_strategy`: AL strategy selection [coreset|stratified|diversity|hybrid|hierarchical|attribute]
- `--al_embedding_dim`: Feature vector dimension (default: 512)
- `--al_initial_labeled`: Initial labeled sample count (default: 500)

Hybrid Strategy Parameters

- `--al_coreset_ratio`: Core-set sampling ratio (default: 0.4)
- `--al_stratified_ratio`: Stratified sampling ratio (default: 0.4)
- Remaining portion automatically allocated to random sampling

Diversity Strategy Parameters

- `--al_clusters`: Number of clusters (default: n_select//10)

Attribute Strategy Parameters

- `--attribute_file`: Path to attribute metadata JSON (e.g., `save/attribute10.json`)
- `--al_attribute_focus`: Optional comma-separated attribute keys to prioritize (default: all keys in metadata)
- `--al_attribute_rarity_power`: Emphasis on rare attribute values (>=0, higher = stronger focus on rare slices)
- `--al_attribute_uncertainty_boost`: Score bonus for samples marked as uncertain in metadata (>=0)
- `--al_attribute_use_entropy`: If True, computes Information Entropy from model inference and uses it for the U term instead of static metadata flags (default: False)
- `--al_attribute_log_path`: Optional path to diagnostic log providing Rare Case accuracies
- `--al_attribute_log_acc_scale`: Scaling factor applied to each log-derived failure weight (default: 1.0)
- `--al_attribute_extra_counts`: Inline list or JSON file describing additional counts to inject per attribute value (useful when synthetic data floods specific slices); inline entries without `=count` inherit `--num_added_img`, and when omitted entirely the code infers counts from `--attributes` tokens (value+attribute) using `--num_added_img`
- `--al_attribute_balance_strength`: Multiplier controlling how strongly over-represented values are penalized (0 disables the term)
- `--al_attribute_class_balance`: Stratified weight for enforcing class balance after attribute scoring (0 disables, `inf` enforces near-perfect round-robin per class)

Recommended Configurations

For ImageNet Dataset (Multi-class Classification)

```
# Recommended: Hybrid strategy, balancing performance and efficiency
python alpred.py \
  --dataset imagenet --model resnet \
  --al_strategy hybrid \
  --al_coreset_ratio 0.2 \
  --al_stratified_ratio 0.6 \
  --al_embedding_dim 256 \
  --sub_ratio 0.5

# Alternative: Pure stratified sampling, fastest and most stable
python alpred.py \
  --dataset imagenet --model resnet \
  --al_strategy stratified \
  --sub_ratio 0.5
```

For CelebA Dataset (Binary Classification)

```
# Recommended: Diversity strategy
python alpred.py \
  --dataset celeba --model resnet \
  --al_strategy diversity \
  --al_clusters 50 \
  --al_embedding_dim 128 \
  --sub_ratio 0.5
```

Performance Expectations

Based on analysis, expected performance ranking:

1. **StratifiedStrategy**: Most stable, maintains class balance
2. **HybridStrategy**: Best overall performance
3. **DiversityStrategy**: Moderate performance, adapts to complex distributions
4. **CoreSetStrategy**: Possibly worst performance (original issue)

Testing Scripts

Run comparative tests of all strategies:

```
./test_al_strategies.sh
```

View result comparisons:

```
python compare_al_results.py
```

Debugging Suggestions

1. **Use –verbose True** to view detailed class distribution information
2. **Start with small datasets** to verify strategy functionality
3. **Compare training times** to select strategy suitable for your computational resources
4. **Check class balance** to ensure selected samples have sufficient representativeness

Code Structure

New AL strategies are all in `utils/active_learning.py`: - `CoreSetStrategy`: Original core-set method - `StratifiedStrategy`: Stratified sampling - `UncertaintyStrategy`: Uncertainty-based sampling (future extension) - `DiversityStrategy`: Clustering-based diversity sampling - `HybridStrategy`: Mixed multiple strategies

Each strategy is an independent class that can be easily extended and modified.

6. Improved Strategies (New)

I have implemented two robust strategies to correct the weaknesses of previous methods (ignoring class balance or ignoring semantic rarity).

A. Uncertainty-Weighted Stratified Strategy (`uncertainty_stratified`)

```
python alpred.py --al_strategy uncertainty_stratified
```

Motivation: Standard uncertainty sampling often picks outliers from a single “hard” class, causing imbalance. Standard stratified sampling ignores sample difficulty. This strategy fuses them.

Algorithm: 1. **Inference**: Compute uncertainty for all unlabeled samples. * *Multi-class*: Entropy $H(p) = -\sum p_i \log p_i$. * *Binary*: Confidence gap $1 - |p - 0.5| \times 2$. 2. **Quota Calculation**: Determine strict sample quotas for each class c: $k_c \approx N_{total}/C$. 3. **Selection**: * For each class c, retrieve all samples predicted as c (or labeled as c if labels available). * Sort them by uncertainty (High to Low). * Select top k_c samples.

Key Benefit: Guarantees diversity across classes (like Stratified) whilst maximizing information gain (like Uncertainty).